

Sueños al Horno — Planificación — Monolito API

Contexto

- **Proyecto:** Sueños al Horno
 - **Clienta:** Sugey Godoy
 - **Paquete:** Torre
 - **Indicaciones:** MHENRIQUEZ solicita **reiniciar desde cero** el avance previo y **reestructurar** hacia un **Monolito API**.
-

1) Análisis técnico y arquitectura: Monolito API

¿Qué implica convertir *Sueños al Horno* en un Monolito API?

En la arquitectura anterior de MHenriquez C.A. existía una integración estrecha con **Livewire 4 (Volt)**. Al transformarlo en un **Monolito API** bajo **Laravel 13**, se propone:

1. **Separación clara de responsabilidades dentro de una sola aplicación (Monolito)**
 - **Backend / API First:** Laravel expondrá un sistema robusto de **APIs RESTful** completas y documentadas (endpoints para **Productos, Blog, Suscriptores/Newsletter, Autenticación** y **Panel Admin**).
 - **Consumo Frontend unificado / modular:** El frontend (**Tailwind CSS v4 + Alpine.js / componentes interactivos**) consumirá estas APIs internas, y permitirá en el futuro conectar apps móviles/externas sin reescribir la lógica de negocio.
2. **Autenticación y seguridad**
 - Implementación de **Laravel Sanctum** (tokens por sesión/bearer) para gestionar sesiones del **Panel Admin** y proteger rutas con **Spatie Permission**.
3. **Mantenimiento del estándar corporativo**

- **Stack:** PHP 8.3, Laravel 13, Tailwind CSS v4, Alpine.js, MySQL (SQLite en memoria para tests con Pest).
 - **Servicios externos:** Resend API para e-mails de doble opt-in.
-

2) Nueva planificación de desarrollo (Roadmap por Sprints)

Metodología **Scrum** con **Sprints numerados**, priorización con **Matriz de Covey** y **commits semánticos en español** (estándar MHenriquez C.A.).

Flujo: Sprint 1 → Sprint 2 → Sprint 3 → Sprint 4

- Sprint 1: Core API & Auth
 - Sprint 2: Core Módulos
 - Sprint 3: Frontend & UI
 - Sprint 4: Deploy & QA
-

Sprint 1 — Inicialización limpia, base de datos y Core API (*Q1 — Urgente*)

Objetivo: Reiniciar el repositorio desde cero, establecer la estructura base del Monolito API y la autenticación.

Backlog del sprint (Issues)

1. **chore:** Reiniciar repositorio desde cero con Laravel 13 + PHP 8.3 + Pest.
 2. **feat:** Configurar Laravel Sanctum para autenticación de API.
 3. **feat:** Crear migraciones y modelos base (**User** , **Role** , **Permission** con Spatie, **Product** , **Post** , **Subscriber**).
 4. **feat:** Implementar **RoleSeeder** y **AdminSeeder** .
 5. **feat:** Endpoints de autenticación:
 - **POST** /api/v1/login
 - **POST** /api/v1/logout
 - **GET** /api/v1/me
 6. **test:** Configurar suite de pruebas Pest (SQLite en memoria) para Auth API.
-

Sprint 2 — Desarrollo de módulos & Endpoints API (Q2 — *Importante*)

Objetivo: Construir los endpoints RESTful para los 4 módulos del sistema (Catálogo, Blog, Newsletter y Panel Admin).

Backlog del sprint (Issues)

2.1 Módulo Catálogo

- **feat:** Endpoints públicos:
 - `GET /api/v1/products`
 - `GET /api/v1/products/{id}`
- **feat:** Endpoints admin CRUD (protegidos por permisos):
 - `POST /api/v1/admin/products`
 - `PUT /api/v1/admin/products/{id}`
 - `DELETE /api/v1/admin/products/{id}`

2.2 Módulo Blog

- **feat:** Endpoints públicos:
 - `GET /api/v1/posts`
 - `GET /api/v1/posts/{slug}`
- **feat:** Endpoints admin CRUD para publicaciones.

2.3 Módulo Newsletter (Doble opt-in)

- **feat:** `POST /api/v1/newsletter/subscribe` (envío de mail de confirmación vía Resend API).
- **feat:** `GET /api/v1/newsletter/confirm/{token}` para activar suscripción.

2.4 Pruebas

- **test:** Cobertura de tests Pest para endpoints CRUD y flujo de doble opt-in.

Sprint 3 — Frontend, integración API y UI (Q2 — *Importante*)

Objetivo: Construir las vistas públicas y el panel de administración que consumen los endpoints de la API.

Backlog del sprint (Issues)

1. **style:** Configuración de Tailwind CSS v4, Alpine.js y paleta de colores (corporativa o del cliente).
 2. **feat:** Catálogo público interactivo con filtros (consumiendo API `/products`).
 3. **feat:** Sección de Blog y detalle de recetas/novedades (consumiendo API `/posts`).
 4. **feat:** Formulario de captura para Newsletter con feedback interactivo en Alpine.js.
 5. **feat:** Panel Admin:
 - Dashboard
 - Gestión de productos
 - Gestión de posts
 - Lista de suscriptores (consumiendo API admin)
-

Sprint 4 — Pruebas finales, CI/CD y despliegue (Q1/Q2)

Objetivo: Validar estándares técnicos, ejecutar build y dejar el sistema desplegado en producción.

Tareas / Issues

1. **test:** Ejecutar batería completa de pruebas automatizadas con Pest (`php artisan test`).
 2. **chore:** Configuración de `deploy.php` y Webhook de CI/CD con verificación **HMAC-SHA256**.
 3. **chore:** `npm run build` para compilación de assets en `public/build/` (estándar MHenriquez C.A.).
 4. **docs:** Actualización completa de la documentación interna y el `README.md` .
-

Flujo de trabajo semántico confirmado

- **Ramas:** `feature/`, `fix/`, `refactor/`, `chore/`, `docs/`
 - **Commits:** semánticos en español
Ej: `feat: implementar endpoints de catálogo (#N)`
 - **Construcción:** `npm run build` previo a commits en `resources/` (servidor sin Node).
-